What a reward model can show depends on who stopped the episode: failure judges and monitors on a low-cost arm

Ravindu Somawansa
Shisso Robotics, Paris
ravindu.somawansa@shissorobotics.com

Abstract. Reward models that score robot rollouts are also used as failure detectors. We ask what such a detector has to beat on a dataset where operators stop the episodes. We use two SO-101 datasets. ArmnetBench has 2,047 rollouts and no time limit: its operators stop a success when the task is done and let a failure run on. There, episode length alone separates failure from success at AUROC 0.942 within policy, and the better of two zero-shot reward models reaches 0.821. On our own 88 rollouts the operator stopped each run by judgment, and length falls to 0.52, close to chance. A judge that sees the finished episode reaches AUROC 1.00 on 50 of them. The baseline a detector has to beat depends on how the dataset's episodes were stopped.

Code and rows: `github.com/ravediamond/paper-reward-models-stopping-rule` **Reproduce:** `make check`

1 Introduction

Papers that evaluate a reward model as a failure detector normally compare it with other learned detectors. On ArmnetBench the operator stopped each rollout and there was no set time limit. Successes are cut at completion and failures run on, and the length of an episode also encodes its outcome. The identity of the policy that ran can carry it too, and we call this the policy-identity null (mainly with models that give out-of-distribution results). On our own rollouts the operator stopped each run by his judgment, and both signals are weaker. The way a dataset ends its episodes is its stopping rule. None of the reward-model papers in our library reports the stopping rule or a length baseline.

We started with a narrower claim, that elapsed time is the reward model to beat. Our own rollouts did not support it. Length is near chance on them. A robot-trained judge passed its threshold on them. What differs between the two results is how the episodes ended, so the stopping rule became the subject of the paper.

The question of this paper is what a reward model has to beat to count as a failure detector on a dataset where operators stop the episodes. We measured length on ArmnetBench [12] and on our own rollouts, and compared it with two zero-shot reward models, Robometer [5] and TOPReward [1]. On our 88 operator-stopped rollouts we also compared it with every judge and monitor we had run. Every bar and every primary score was fixed before the test it judges.

2 Data and labels

ArmnetBench v0.1 [12] has 8 tasks and 7 policies on the SO-101. We use its 2,047 policy rollouts that have a success or failure class (515 successes, 1,532 failures) and leave out the teleoperated episodes. Robometer scored 2,099 rollouts, the extra ones labelled suboptimal; every AUROC here uses the 2,047. The benchmark sets no time limit. Its operators stop a success when the task is done and let a failure run on, so only 8.7 % of failures reach the longest episode of their task. One task, pushing a Displayport cable (270 rollouts), has rollouts of a single class, so it has no per-task AUROC.

Our own data are 88 labelled episodes of one task, placing one bowl in another, on one SO-101 arm with a Jetson Orin NX. TOPReward scored 69 of them, with 25 failures, 17 of which come from one policy, TurboVLA [16]. Every judge and monitor used the 50 of these 69 that TurboVLA did not run (42 successes, 8 failures). The author was the operator, stopped each run and labelled it at rest, and was not blind to the policy. What these data cannot show: the judge and monitor results rest on 8 failures of one task, and the judges' operating thresholds were chosen on the same 50 episodes.

3 The length baseline

The goal of a failure detector is to rank failed episodes above successful ones, and we report its failure AUROC (0.5 is chance, 1 is perfect). Pooled means all rollouts taken as one set. A pooled AUROC can be high only because some policies fail more often than others, so we also report it within policy: the AUROC inside each policy's rollouts, averaged with each policy weighted by its number of success–failure pairs (pair-weighted; the code is in the appendix). The free baseline is episode length, with longer meaning failure. It is strong on ArmnetBench because of the stopping rule: a success is stopped when it is done and a failure runs on. Intervals are 95 % bootstrap intervals over episodes (2,000 resamples). When a paired interval on a difference lies entirely below zero, the loss is not luck.

On ArmnetBench, episode length is a strong failure detector within each policy (Table 1, Fig. 2). Pooled over policies it reaches 0.938. Within each policy it ranges

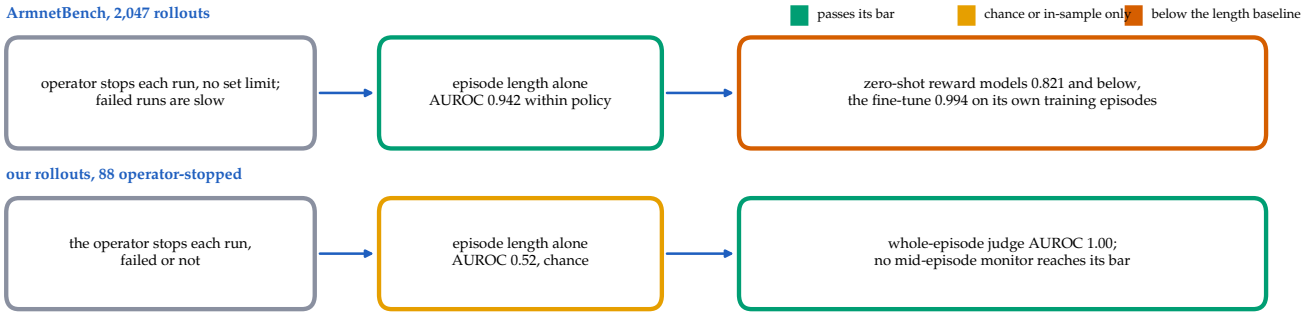


Figure 1: **What the baseline is depends on how the episodes ended.** ArmnetBench (operator-stopped, no set limit): length 0.942 within policy, both zero-shot reward models below it, the fine-tune only in-sample. Our operator-stopped rollouts: length 0.52, chance; a whole-episode judge 1.00; no mid-episode monitor reaches its bar.

signal	method as	pooled	within policy
episode length	free baseline	0.938 (0.925–0.951)	0.942 (0.927–0.956)
length seen up to 50 % of the longest run	free baseline	—	0.851 (0.829–0.873)
Robometer-4B, success probability	authors', as shipped	0.818 (0.790–0.843)	0.821 (0.794–0.847)
Robometer-4B, last progress (descriptive)	authors', as shipped	0.838 (no CI)	—
Robometer-4B fine-tuned on ArmnetBench, success probability (in-sample)	released fine-tune	0.994 (0.991–0.996)	0.994 (0.992–0.997)
TOPReward, success score	authors', fitted	0.653 (0.623–0.684)	0.661 (0.631–0.692)
TOPReward, Yes/No component (descriptive)	authors', fitted	—	0.678

Table 1: ArmnetBench, failure AUROC with 95 % episode-bootstrap CIs, on the 2,047 rollouts. Pooled: all rollouts as one set; within policy: pair-weighted mean of the per-policy AUROCs. Read down the within-policy column: no zero-shot reward-model score reaches length, and the fine-tune row is in-sample.

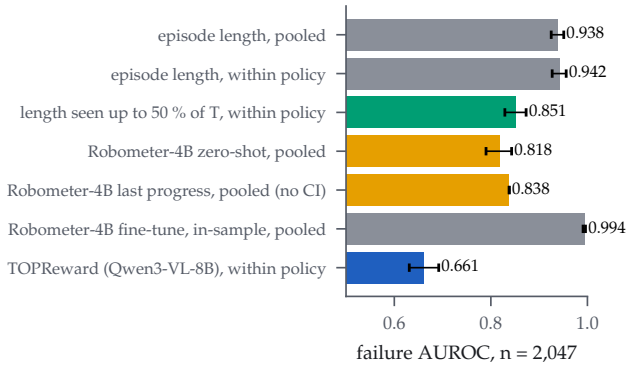


Figure 2: Failure AUROC on ArmnetBench: episode length against two zero-shot reward models. Length leads both, pooled and within policy (95 % CIs where available).

from 0.91 to 0.96. Pair-weighted, the within-policy value is 0.942, with a CI of 0.927–0.956.

We ran Robometer-4B [5] zero-shot, as its authors ship it (the authors' method). It scores 0.818 pooled and 0.821 within policy (CI 0.794–0.847). Its paired difference to length within policy has a 95 % CI of -0.150 to -0.094, entirely below zero. We ran TOPReward [1] with Qwen3-VL-8B [11] and no training. Its paper does not give the number of prefixes, the frames per query or the wording of the Yes/No question, so we chose them, and we count it as the authors' method fitted to our data. It scores 0.661 within policy. Its paired difference to length has a 95 % CI of -0.31 to -0.25. The primary score is the success probability for Robometer and the success score of its paper for TOPReward. Two other scores from the same runs are in Table 1 as descriptive rows. They are higher, and still below length.

We also scored the released ArmnetBench fine-tune of Robometer, `jesbu1/robometer-4b-fft-armnet-tiled`, since the obvious reply to a zero-shot result is to use it. Its model card lists a dataset of all 2,499 ArmnetBench episodes, with no held-out split, as its training data. Every episode we score was therefore a training episode with its label, and the result is in-sample. On the 2,047 rollouts it scores 0.994 pooled and 0.994 within policy, 0.052 above length (paired CI 0.039 to 0.067). This shows that the labels can be fit. It does not show that a reward model beats length on rollouts it has not seen. We gave it 8 frames spread over the 64 of each tiled video, the length it was trained on. Scoring all 64 frames on 160 episodes gives 0.995 pooled against 0.989 for 8 frames.

4 Length is more than the longest episode

Only 8.7 % of failures run as long as the task's longest episode, which we use as a reference length (the benchmark sets no time limit). At half that length, 88 % of

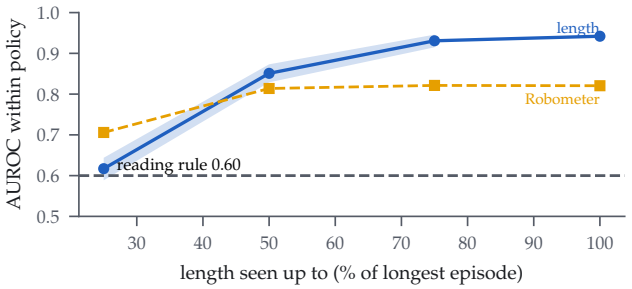


Figure 3: Length observed up to a fraction of the task's longest episode, AUROC within policy: most of the signal is there by half the episode (band: 95 % CI).

failures are still running, and only 19 % of successes are. Length observed up to that cut gives 0.851 within policy (Fig. 3). The CI is 0.829–0.873. TOPReward on frames up to the same cut reaches 0.667. Robometer's success probability at the last frame before the cut gives 0.814 within policy (CI 0.787–0.839), which is below length at that cut. At a quarter of the longest episode the order is the other way, 0.706 for Robometer against 0.617 for length, and by three quarters length is ahead again, 0.931 against 0.821. These are prefixes of one pass over the whole episode (§7). Failed rollouts are slow; most do not reach the task's longest episode.

Own rollouts. On our own operator-stopped rollouts, length is at chance. It gives 0.56 on the 69 rollouts TOPReward scored and 0.52 on all 88. Here the policy-identity null is the baseline to beat: it scores each episode by the failure rate of the policy that ran it, on the same episodes. It is strong because 17 of the 25 failures in the 69 come from one policy, TurboVLA [16]. Neither reward model beats policy identity. TOPReward scores 0.82 (CI 0.70–0.93), and policy identity alone 0.87. On the 50 episodes without TurboVLA, TOPReward scores 0.78. Robometer scores 0.577 on all 88, and policy identity 0.878.

5 Judges and monitors on our rollouts

A judge looks at the finished episode and says whether it succeeded. A monitor reads the episode while it runs and must raise an alarm before the end, early enough for the robot to stop. Table 2 gives each a threshold-free AUROC where one is defined, and the failures caught and successes flagged at its operating point, the score threshold that turns scores into flags. For the judges that point was chosen on the same 50 episodes (in-sample), so their counts are optimistic. The monitors' thresholds were fixed before scoring.

We first ran a deterministic top-camera check, with colour masks and geometry and no learning. It is right on 94.3 % of judged episodes. It has 0 false positives and 5 false negatives. The Wilson interval on the false positives is 0–13 then gave Flora-Rewarder-27B [8] a 64-frame two-view clip and NVIDIA's prompt. Its threshold-free AUROC is 1.00. The bootstrap CI is 1.00–1.00. At its in-sample operating point it flags 8 of 8 failures at 2 of 42 successes. The Wilson interval on those false alarms is 1–16 at least 6 of 8 failures at no more than 2 of 42 successes, so it passes. Its base model, Qwen3.5-27B [10], has no robot fine-tune. On the same input it also passes, with AUROC 0.91 and 7 of 8 at 2 of 42. The interval on that AUROC is 0.78–1.00. The input format mattered more than the fine-tune. Qwen3.5-9B [10] flags 16 successes and fails. Gemini 3.8 Flash [2] fails by one false alarm.

The bar for mid-episode monitors is at least 5 of 8 failures at no more than 1 of 20 false interventions. No monitor reaches it on these 50 rollouts (Fig. 4). The surprise of a separate world model, trained in the LeWorldModel recipe [7], catches 2 of 8 at 2 of 42. Its AUROC is 0.86. A Gemini sliding-window judge catches 0. The first action-conditioned risk head on the world model's latent reached AUROC 0.93. It read elapsed time, so we withdrew that result. We built a second version without time or length. It catches 3 of 8 at 5 of 42. Its AUROC is 0.79, and 0.71 with shuffled actions. The MLP variant catches 6 of 8 and flags 13 of 42.

Two comparisons were added after these tests and are exploratory, with no bar. On the 50 episodes the free baselines are weaker than on the 69, because TurboVLA did not run any of them: policy identity gives 0.65 and length 0.57. Flora's AUROC is 0.35 above policy identity, with a paired CI of 0.12 to 0.40, entirely above zero; for Qwen3.5-27B the interval, -0.06 to 0.34, contains zero. The risk head is our own design after ReAP [6]. ReAP releases no code, so we also rebuilt its Stage 1 detector and Stage 2 predictor from the paper, with a smaller DINOv2, one camera, training data mostly from other tasks (ArmnetBench) and one seed of the five planned. It catches 2 of 8 at 1 of 42, AUROC 0.72, and fails the monitor bar. On our side: 8 failures, one task, one seed and a smaller backbone than the paper's.

6 Related work

Robometer [5] scales a general-purpose reward model through trajectory comparisons. TOPReward [1] reads the token probabilities of a video-language model as hidden rewards. A survey of verifiers for robot policies [14] covers the wider family. FAIL-Detect [17] and SAFE [3] learn uncertainty- and feature-based failure scores for imitation and VLA policies. FARM [9] reads failure signals from the internal states of a frozen world model, like our world-model monitors. Our risk head follows ReAP [6], which predicts action-conditioned risk. VLA-Corrector [13] verifies stages, Spotter [4] adds a reflecting VLM, and REMORA [15] tracks progress through errors and recovery. ArmnetBench [12] and a standardized SO-101 VLA benchmark [18] evaluate policies on the same arm. We found no report of the elapsed-length baseline under the dataset's stopping rule in any of them.

method	method as	kind	AUROC	caught / 8	flagged / 42	threshold
policy identity	free baseline	at rest	0.65	—	—	—
episode length	free baseline	at rest	0.57	—	—	—
colour judge (no learning)	ours	at rest	— (94.3 % right, 0 FP, 5 FN)	—	—	fixed rule
Flora-Rewarder-27B	authors' prompt	at rest	1.00 (1.00–1.00)	8	2	in-sample
Qwen3.5-27B, same input	base model	at rest	0.91 (0.78–1.00)	7	2	in-sample
Qwen3.5-9B, same input	base model	at rest	0.75	7	16	in-sample
Gemini 3.8 Flash, same input	base model	at rest	0.96	8	3	in-sample
Gemini sliding windows	ours	monitor	—	0	1	fixed before scoring
world-model surprise	ours, LeWorldModel recipe	monitor	0.86	2	2	fixed before scoring
world-model latent drift	ours, LeWorldModel recipe	monitor	0.30	0	2	fixed before scoring
risk head, logistic (v2)	ours, after ReAP	monitor	0.79	3	5	5-fold by episode
risk head, MLP (v2)	ours, after ReAP	monitor	—	6	13	5-fold by episode
ReAP Stage 1 + 2, rebuilt	authors', reduced	monitor	0.72	2	1	one global threshold

Table 2: Judges and monitors on the 50 operator-stopped rollouts (42 successes, 8 failures). Judges see the finished episode; monitors are held to $\geq 5/8$ caught at $\leq 1/20$ false interventions. "Method as": whose method ran; the free baselines and the ReAP rebuild were added after the tests (exploratory).

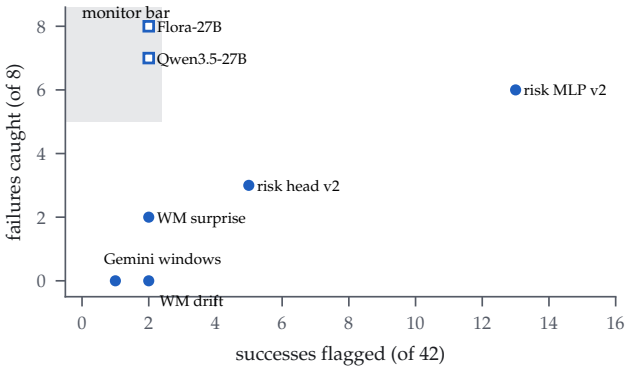


Figure 4: Failures caught against successes flagged on the 50 rollouts. No monitor (circles) reaches its shaded bar; the judges at rest (squares) sit at their in-sample operating point, where the monitor bar does not apply.

7 Limitations

The judge pass on our own rollouts rests on 8 failures. Its operating thresholds were chosen on the same 50 episodes. A single operator, the author, labelled them and was not blind to the policy. The threshold-free AUROCs do not depend on the threshold. Fresh rollouts scored at the same thresholds would confirm the operating points. 17 of the 25 failures in our 69 rollouts are TurboVLA's. Any score that reads the policy therefore gets the policy-identity null for free. The risk head is trained and tested on the same 50 episodes. Five folds by episode keep episodes from leaking, but tasks and policies leak. The fine-tune comparison is in-sample, because the release has no held-out episodes, and its config needed three training-only keys removed to load in the pinned code. Robometer's truncated scores are prefixes of one pass over the whole episode. A frame's value may depend on later frames, so a pass over a clip that ends at the cut could give a different number. The 462 episodes that ran out of memory in the first pass were scored in a second pass on a larger GPU, with every frame kept. TOPReward's paper does not give its prefix count, frames per query or Yes/No wording, so those are our choices. The ArmnetBench result covers eight tasks. The reference length is each task's longest episode; the benchmark sets no time limit.

8 Conclusion

Conclusion. On ArmnetBench, where the operator stopped each rollout and there was no set time limit, episode length beat both zero-shot reward models, and a fine-tune beat it only on the episodes it was trained on. On our operator-stopped rollouts length was at chance, and a judge that saw the whole finished episode caught the failures. When you evaluate a reward model as a failure detector, report episode length within policy on the same rows and say how the episodes ended.

References

[1] Shirui Chen, Cole Harrison, Ying-Chun Lee, Angela Jin Yang, Zhongzheng Ren, Lillian J. Ratliff, et al. Topreward: Token probabilities as hidden zero-shot rewards for robotics. *arXiv preprint arXiv:2602.19313*, 2026.

[2] Google DeepMind. Gemini 3.8 Flash. Google model page, 2026. URL `https://aistudio.google.com/docs/models`. model id gemini-3.8-flash; the page gives no release date; accessed 2026-11-04.

[3] Qiao Gu, Yuanliang Ju, Shengxiang Sun, Igor Gilitschenski, Haruki Nishimura, Masha Itkina,

and Florian Shkurti. Safe: Multitask failure detection for vision-language-action models. *arXiv preprint arXiv:2506.09937*, 2025.

[4] Long Li, Qichao Zhao, Yue Yang, et al. Spotter: Let the embodied model lead, and the vlm reflect for it. *arXiv preprint arXiv:2609.36808*, 2026.

[5] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, et al. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons. *arXiv preprint arXiv:2603.02115*, 2026.

[6] Jingyuan Liu, Yunpeng Qing, Changxu Wei, and Zhi Hou. Reap: From visual failure recognition to action-conditioned risk prediction for runtime failure monitoring. *arXiv preprint arXiv:2610.16309*, 2026.

[7] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[8] NVIDIA. Flora-Rewarder-27B. Hugging Face model card, 2026. URL https://huggingface.co/nvidia/Flora-Rewarder. nvidia/Flora-Rewarder, released 2026-10-15, base model Qwen3.5-27B; NVIDIA OneWay Noncommercial License; accessed 2026-11-04.

[9] Haoran Pei, Mingrui Luo, Senbao Wang, et al. Farm: Reading failure signals from the internal predictive states of a frozen robotic world model. *arXiv preprint arXiv:2609.11445*, 2026.

[10] Qwen team. Qwen3.5-27B and Qwen3.5-9B. Hugging Face model cards, 2026. URL https://huggingface.co/Qwen/Qwen3.5-27B. the 9B card is at huggingface.co/Qwen/Qwen3.5-9B; Apache-2.0; accessed 2026-11-04.

[11] Qwen team. Qwen3-VL-8B. Hugging Face model card, 2026. URL https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct. Qwen3-VL-8B-Instruct, TOPReward's backbone as run here; Apache-2.0; accessed 2026-11-04.

[12] Praveen Selvaraj, Lorenzo Uttini, and Ville Kuosmanen. Armnetbench v0.1: Parallel real-world evaluation of manipulation policies on a low-cost arm farm. *arXiv preprint arXiv:2607.24481*, 2026.

[13] Chang Song, Bin Qian, Yan Feng, and Zhijie Song. Vla-corrector: Stage-aware observable state understanding for prompt-based closed-loop recovery of vision-language-action policies. *arXiv preprint arXiv:2609.06508*, 2026.

[14] Yang Wan, Xihang Yue, Zhirui Liu, Ziyuan Chu, Shuxun Wang, Yuhan Chen, et al. No free checker: A survey of verifiers for robot policies. *arXiv preprint arXiv:2609.09250*, 2026.

[15] Chen Wang, Xinze Niu, Zihao Zhu, Keru Zhou, Yicheng Xu, Shihao Jin, et al. Tracking task progress through errors and recovery: A benchmark and recurrent distributional model. *arXiv preprint arXiv:2610.39125*, 2026.

[16] Hengyi Xie, Chenfei Yao, Xianjin Wu, Yingying Zhu, Dingkang Liang, Xiang Bai, et al. Turbovla: Real-time vision-language-action model at 32 hz on an rtx 4090 with <1 gb vram. *arXiv preprint arXiv:2607.27205*, 2026.

[17] Chen Xu, Tony Khuong Nguyen, Emma Dixon, et al. Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies. *arXiv preprint arXiv:2503.08558*, 2025.

[18] Yi Yu and Xinchuan Qiu. Benchmarking vision-language-action models on so-101: Failure and recovery analysis. *arXiv preprint arXiv:2606.08881*, 2026.

Appendix: the within-policy AUROC

The within-policy value pair-weights the per-policy AUROCs (`repro/length-null/scripts/length_analysis.py`):

```
def stratified(df, score, by="policy_type"):
    """Pair-weighted mean of per-stratum AUROCs (weights successes
      x failures)."""
    num = den = 0.0
    for _, g in df.groupby(by):
        a = auroc(g.y, g[score]); w = g.y.sum() * (~g.y).sum()
        if a is not None:
            num += a * w; den += w
    return num / den if den else None
```

The code, the expected result files and the protocol of every analysis are in the repository; its README maps each analysis to its files.